UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE LOS VALLES

Lic. Tecnologías de la información

Materia

Desarrollo de Bases de Datos para web

Actividad.

U1 – A5 Proyecto de la unidad 1

Alumno.

Miguel Ángel Berumen Aguilar

Profesor.

Adolfo Castillo Chavarin

Fecha: 28/09/2026

Introducción

En esta actividad desarrollé un mini-sistema web modular aplicando el patrón de arquitectura Modelo-Vista-Controlador (MVC) con el microframework Flask y MySQL Community Server. El objetivo principal fue integrar vistas dinámicas con Jinja2, formularios protegidos mediante Flask-WTF y operaciones CRUD (Crear, Leer, Actualizar y Eliminar) para administrar cuentas de usuario dentro de una base de datos relacional.

A lo largo de la práctica apliqué conceptos clave de autenticación básica basada en sesiones, validación de datos de entrada en el servidor y protección contra ataques CSRF, logrando comprender el flujo de información completo desde que el usuario

interactúa con la interfaz web hasta que las consultas se procesan en la base de datos.

ARQUITECTURA DEL SISTEMA Y DECISIONES DE DISEÑO

Para estructurar la aplicación de forma limpia y mantenible, dividí el código en capas según el patrón MVC exigido en la materia:

Modelo (models/user.py): Definí la entidad User utilizando SQLAlchemy para conectarme a MySQL (db_proyecto1). La estructura representa la tabla de usuarios con los campos id, username, email y password.

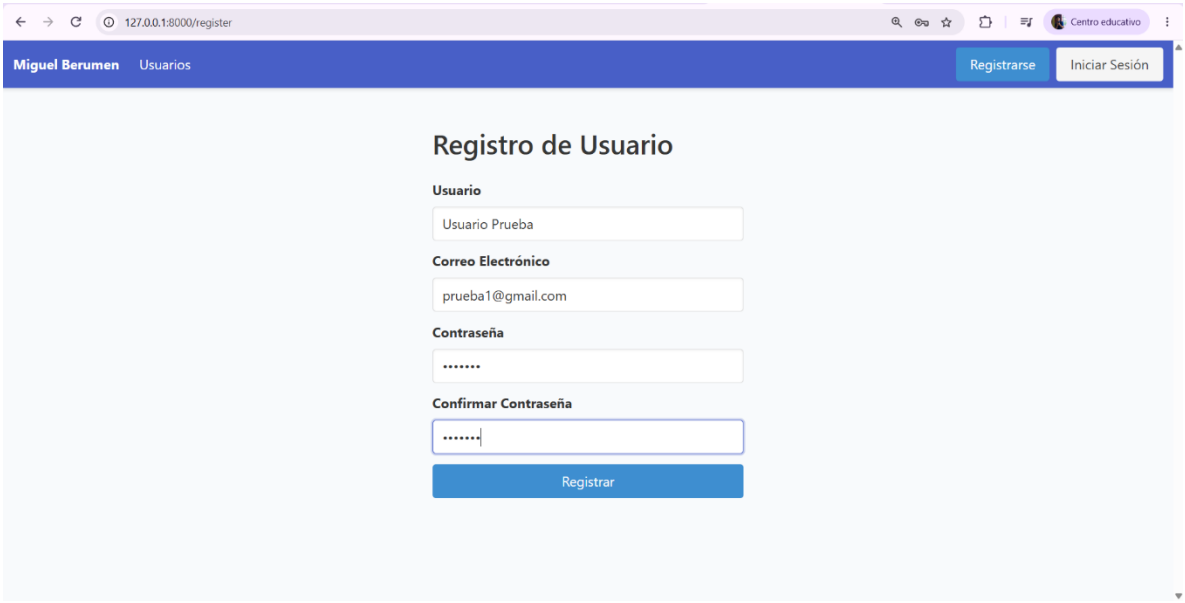
Controlador y Rutas (routes/user_routes.py): Implementé un Blueprint (user_bp) para agrupar todas las rutas de la aplicación (/, /register, /login, /logout, /profile/, /profile//edit, /profile//delete). Creé además un decorador personalizado @login_required utilizando functools.wraps para proteger las rutas privadas mediante el objeto session de Flask.

Formularios y Seguridad (forms.py): Utilicé Flask-WTF para construir los formularios con sus respectivas reglas de validación (DataRequired, Email, Length, EqualTo), asegurando que la información sea correcta antes de procesarla en la base de datos y manteniendo activa la protección CSRF.

Vistas y Estilos (templates/ y static/): Construí la interfaz extendiendo una plantilla maestra (base.html) e incluyendo fragmentos como _messages.html para desplegar alertas mediante flash(). Apliqué estilos visuales con Bulma CSS y un archivo static/style.css personalizado.

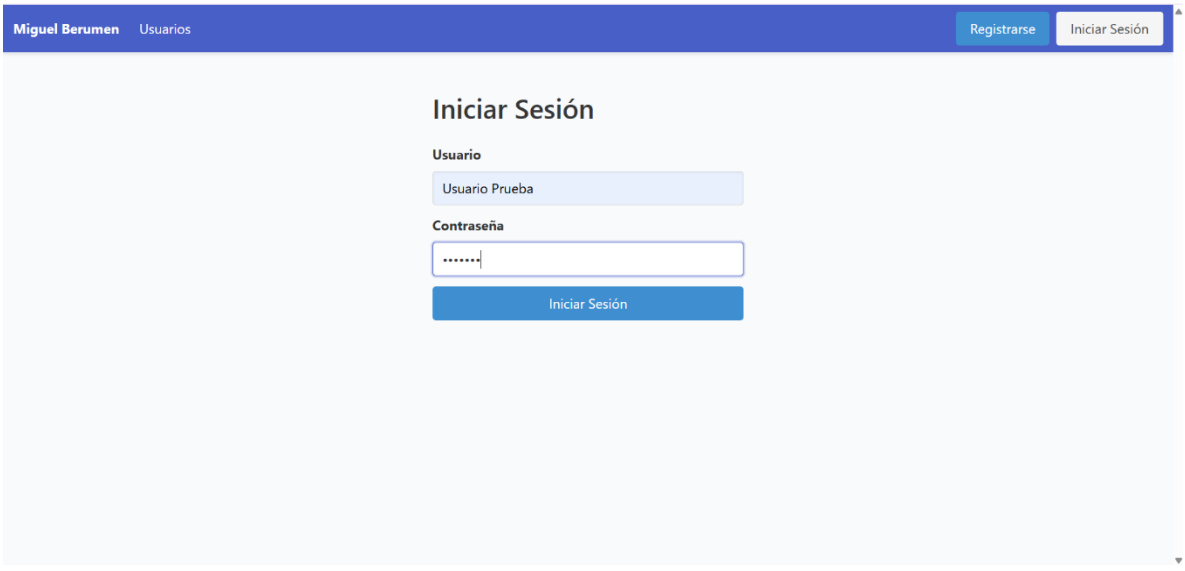
Capturas de pantalla

Registro de usuarios



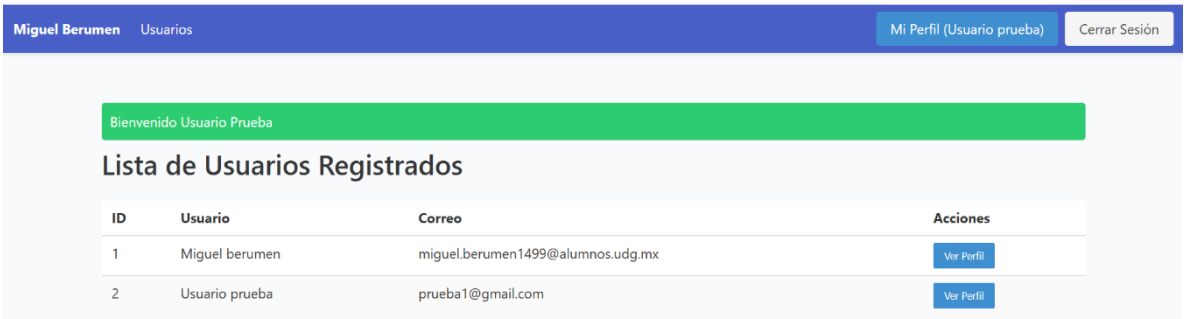
En esta parte hice el formulario para registrar nuevos usuarios. El usuario tiene que ingresar su nombre, correo y contraseña, además de confirmar la contraseña. También agregué validaciones para evitar que se registre un usuario o correo que ya exista en la base de datos. Si esto sucede, el sistema muestra un mensaje indicando el problema y no guarda los datos repetido

Formulario de Login



Aquí desarrollé el formulario para que los usuarios puedan iniciar sesión. El sistema verifica que el correo y la contraseña coincidan con los datos que están guardados en MySQL. Cuando los datos son correctos, se guarda la información del usuario en la sesión y se le permite acceder a las partes privadas de la aplicación.

Lista general de usuarios



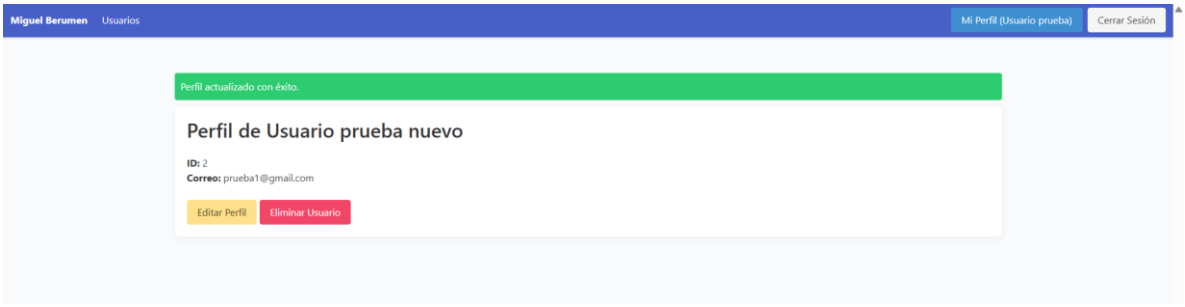
Esta es la pantalla principal donde se muestran los usuarios que están registrados en la base de datos. Para obtener la información utilicé SQLAlchemy y después mostré los datos en una tabla. También utilicé el filtro |capitalize de Jinja2 para que los nombres se mostraran con la primera letra en mayúscula. Además, después de iniciar sesión aparece un mensaje de bienvenida usando flash().

Pantalla de Perfil



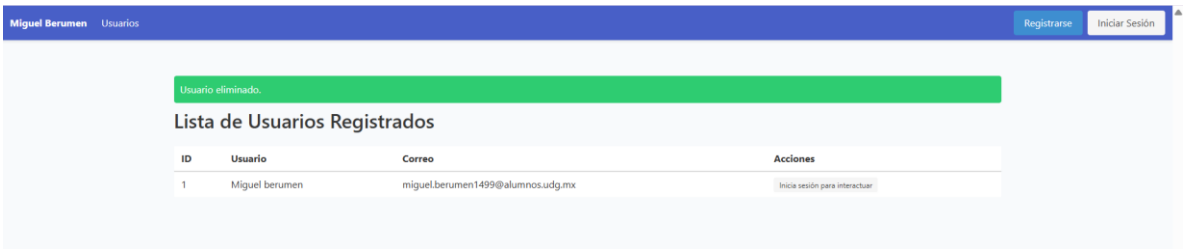
En esta sección se puede consultar la información de un usuario específico. La ruta está protegida con @login_required, por lo que es necesario haber iniciado sesión para poder acceder. Desde aquí también agregué las opciones para editar los datos del usuario o eliminar su cuenta.

Formulario de edición



En esta parte hice el formulario para modificar los datos de un usuario. Al entrar, el formulario muestra la información que ya estaba registrada y permite cambiarla. Cuando se envían los nuevos datos, el sistema actualiza la información en la base de datos y muestra un mensaje indicando que la modificación se realizó correctamente.

Confirmación de eliminación



En esta sección agregué la opción para eliminar un usuario de la base de datos. Cuando se confirma la eliminación, el sistema realiza el borrado en MySQL. Si el usuario elimina su propia cuenta, también se utiliza session.clear() para cerrar su sesión y después se redirige al listado general. Finalmente, se muestra un mensaje para indicar que la cuenta fue eliminada correctamente.

CONCLUSIONES

El desarrollo de este mini-sistema me permitió consolidar los conocimientos sobre el patrón MVC y el manejo de bases de datos relacionales en entornos web con Python. Aprendí la importancia de desacoplar las vistas de la lógica de negocio usando Blueprints y cómo gestionar sesiones de usuario de forma segura.

Este proyecto me brinda una base sólida para abordar aplicaciones web más complejas en los siguientes bloques de la materia.